GEN AI & AGENTIC AI COURSE GUIDE

Module 1

Generative AI Foundations

A detailed, illustrated walkthrough of generative vs. discriminative models, transformer architecture, tokenization, how LLMs are trained, the model landscape, and a hands-on lab.

Prepared for Roja · September 2026

1. Generative vs. Discriminative Models

The deep learning work you just finished — Haar cascades for face/eye detection, OCR with pytesseract and EasyOCR, color detection — is all **discriminative**. A discriminative model looks at existing data and answers a question about it: is there a face here? What character is this? Discriminative models learn a boundary between categories.

A **generative** model does the opposite: instead of labeling data that already exists, it learns the underlying patterns well enough to produce brand-new data that looks like it belongs to the same distribution — a new sentence, a new image, a new block of code. An LLM completing a sentence is a generative model estimating "given everything so far, what word is most likely to come next?" and sampling from that probability distribution, one token at a time.

A simple way to tell them apart

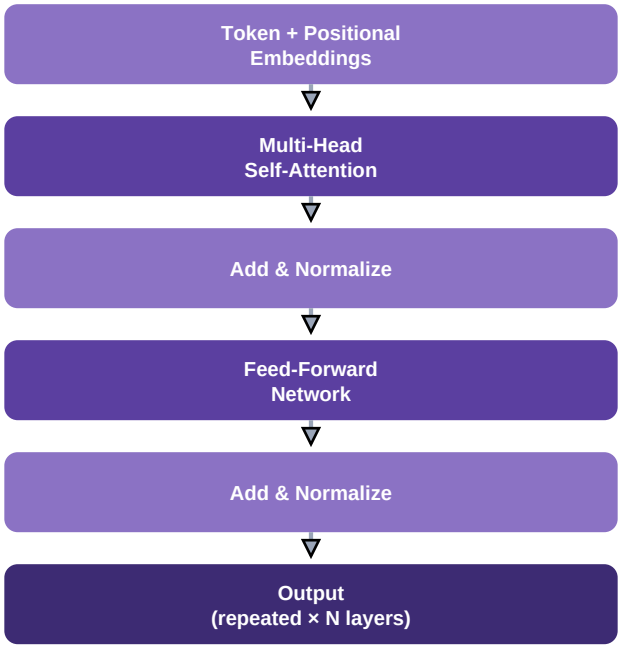
	Discriminative	Generative
Question asked	"What is this?"	"What comes next / what could this be?"
Output	A label or category	New content (text, image, audio, code)
DL-module example	Haar cascade face detector	An LLM writing a paragraph
Learns	A decision boundary	A probability distribution over data

Key takeaways

- Discriminative models classify or detect within existing data.
- Generative models produce new data by learning its underlying distribution.
- An LLM is fundamentally a next-token predictor sampling from a learned distribution.

2. Transformer Architecture

Before 2017, sequence models (RNNs, LSTMs) read text one word at a time, in order. That made them slow to train and forgetful over long passages: information from early in a sentence would fade by the time the model reached the end. The 2017 paper *"Attention Is All You Need"* introduced the **transformer**, which processes every word in a sequence simultaneously and lets each word look directly at every other word. This is the architecture behind every major LLM today, including GPT, Claude, and Gemini.



One transformer block. Real models stack this block many times — e.g. roughly 96 layers in GPT-3-scale models.

2.1 Self-attention

Self-attention answers one question for every word: *which other words in this sentence should I pay attention to, and how much?* Take the sentence: "The cat sat on the mat because it was tired." To understand what "it" refers to, the model needs to connect "it" back to "cat" rather than "mat," even though both are grammatically plausible.

Each word is turned into three vectors, learned during training:

- **Query (Q)** — what this word is looking for in other words.
- **Key (K)** — what this word has to offer, as a label other words can match against.
- **Value (V)** — the actual content this word contributes once it's picked.

The model compares "it"'s Query vector against every other word's Key vector using a dot product — a similarity score. Higher similarity means more relevance. Those scores are turned into weights (via a softmax, so they sum to 1), and the model builds a new representation of "it" as a weighted blend of every word's Value vector.

A toy example with simplified, single-number "vectors" (real ones have hundreds of dimensions): Query for "it" = 0.9. Keys: "cat" = 0.85, "tired" = 0.40, "mat" = 0.10. Turning those similarities into normalized weights gives roughly:

Attention weight from "it" onto each candidate word — "it" ends up carrying mostly "cat."

2.2 Multi-head attention

A word can be relevant to another word for more than one reason at once — grammatically, semantically, positionally. Multi-head attention runs several independent attention calculations ("heads") in parallel, each with its own learned Q/K/V projections, so different heads can specialize: one might track subject-verb agreement, another pronoun references, another topic. Their outputs are concatenated into one final representation per word. GPT-style models typically use 12 to 96+ heads depending on model size.

2.3 Positional encoding

Because self-attention looks at all words simultaneously rather than in sequence, it has no built-in sense of word order — "dog bites man" and "man bites dog" would look identical without extra information. **Positional encoding** injects each word's position directly into its vector, usually as a pattern of sine and cosine values (or, in newer models, rotary position embeddings/RoPE) that the model learns to interpret as "this word is 3rd," "this word is 47th," and so on.

2.4 Encoder, decoder, and encoder-decoder architectures

- **Encoder-only** (e.g. BERT) — reads the whole input at once, in both directions; good for understanding tasks like classification or search, but doesn't generate text token by token.
- **Decoder-only** (e.g. GPT, Claude, Llama) — generates text one token at a time, only ever looking backward at tokens already produced ("causal" attention). This is the architecture behind almost every modern chat-style LLM.
- **Encoder-decoder** (e.g. T5, the original transformer) — an encoder reads the full input and a separate decoder generates the output, attending back to the encoder's representation. Well suited to translation and summarization.

2.5 Why transformers replaced RNNs/LSTMs

- **Parallelization** — RNNs process tokens strictly in sequence, so training can't be parallelized across a sequence. Transformers process a whole sequence at once, dramatically faster on GPUs.

- **Long-range dependencies** — RNNs lose information as sequences get longer. Self-attention gives every token a direct, constant-distance connection to every other token, regardless of how far apart they are.

> **Key takeaways**
>
> - Self-attention lets every token weigh the relevance of every other token, via Query/Key/Value vectors.
> - Multi-head attention runs several attention patterns in parallel to capture different kinds of relationships.
> - Positional encoding restores word-order information that pure attention would otherwise lose.
> - Decoder-only architectures (GPT, Claude, Llama) are the backbone of modern chat LLMs.

3. Tokenization and Context Windows

LLMs don't read whole words. Text is first broken into **tokens** — pieces that are often smaller than a word (a common word might be one token; a rare or technical word might split into two or three sub-word pieces). Tokenization averages roughly 4 characters per token for English.

3.1 Byte-Pair Encoding (BPE)

Most LLMs tokenize using a variant of **Byte-Pair Encoding**. The algorithm starts with individual characters and repeatedly merges the most frequently co-occurring pair into a single new token, building a fixed-size vocabulary (commonly 30,000–100,000+ tokens). Frequent words like "the" become a single token; rarer words split into pieces, for example:

Illustrative sub-word split of "Generative AI transforms e-commerce." — actual splits depend on the specific tokenizer.

This lets the model handle any text, including words it never saw during training, by falling back to smaller sub-word or character pieces.

3.2 Vocabulary size trade-offs

- A **larger vocabulary** means more words are captured as a single token (shorter sequences, cheaper to process) but a bigger embedding table to learn and store.
- A **smaller vocabulary** means more words get split into multiple sub-word tokens (longer sequences, more expensive) but a smaller, more efficient model.

3.3 Context windows

The **context window** is the maximum number of tokens a model can consider at once — prompt plus conversation history plus the response it's generating. A model with a 128k-token context window can hold roughly 90,000–100,000 words of text in play at once (code and non-English text often tokenize less efficiently). Two practical consequences worth remembering:

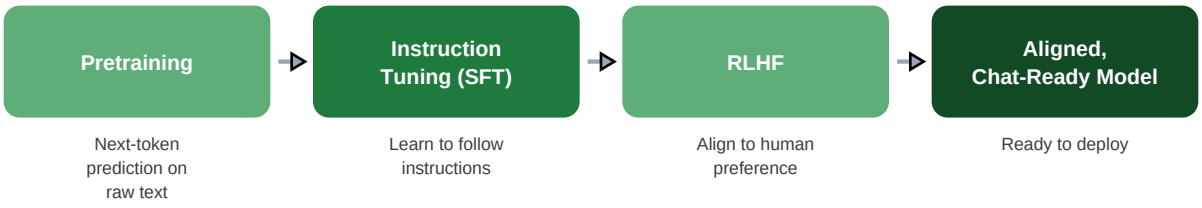
- **Cost** — API pricing is per token (input and output separately), so both tokenization efficiency and context length directly affect what you pay.
- **Compute** — standard self-attention compares every token to every other token, so cost and memory scale roughly with the square of the sequence length, which is why very long context windows are an active engineering challenge.

Key takeaways

- Tokens are sub-word units, not whole words — produced by an algorithm like BPE.
- Vocabulary size trades sequence length against embedding-table size.
- The context window caps how much text a model can consider at once, and drives both cost and compute.

4. How LLMs Are Trained

Building a modern chat-style LLM happens in stages. Each stage changes what the model is optimized to do.



The three-stage pipeline that turns a raw base model into an assistant you can chat with.

4.1 Pretraining

The model is trained on a massive corpus of text (a large fraction of the public internet, books, code, and more) with a simple, self-supervised objective: predict the next token, given everything before it. No human labels are needed — the text itself provides the "correct answer" at each position. This single objective, applied at enormous scale, is what teaches the model grammar, facts, reasoning patterns, and style. This stage produces a **base model**: extremely good at continuing text, but not yet good at following instructions or holding a conversation.

4.2 Instruction tuning (supervised fine-tuning / SFT)

The base model is then fine-tuned on a curated set of example conversations: an instruction paired with a high-quality response. This teaches the model the *format* of being helpful — answering questions directly, following instructions, adopting a system prompt — rather than just continuing text in whatever direction it started.

4.3 RLHF (Reinforcement Learning from Human Feedback)

Humans (or AI systems trained to mimic human judgment) rank multiple model responses to the same prompt from best to worst. A separate **reward model** is trained to predict those rankings. The LLM is then further trained, using reinforcement learning (commonly PPO, or newer variants like DPO), to produce responses the reward model scores highly — nudging it toward being more helpful, honest, and safe. This stage shapes a model's "personality" and its willingness (or refusal) to answer certain requests.

4.4 Alignment, in one sentence

"Alignment" is the umbrella term for all the techniques — instruction tuning, RLHF, and others — used to make a model's behavior match what its developers and users actually want, rather than just what maximizes next-token prediction accuracy.

Key takeaways

- Pretraining (next-token prediction on huge text corpora) builds raw language ability.
- Instruction tuning (SFT) teaches the model to follow instructions and hold a conversation.
- RLHF uses a learned reward model plus reinforcement learning to align outputs with human preference.

5. The Model Landscape

Models fall broadly into two categories, and the difference matters for anything you build:

	Closed / API-only	Open-weight
Examples	GPT family, Claude family, Gemini	Llama, Mistral, and others
Access	Called via a hosted API	Downloaded and run on your own hardware
Customization	Limited to prompting, fine-tuning APIs	Full control — fine-tune, quantize, modify
Cost model	Pay per token used	Pay for compute/hosting instead
Data privacy	Data passes through the provider	Can be run fully offline / on-prem

In practice, most teams start by prototyping against a closed API (fast to get started, no infrastructure) and only move to an open-weight, self-hosted model when cost at scale, data privacy, or deep customization needs justify the extra operational work.

6. Hands-on Lab

Goal: load a small open model locally and see tokenization happen firsthand.

Option A — Hugging Face transformers

- Install: `pip install transformers torch`
- Load a small model's tokenizer, e.g. `gpt2` or `distilgpt2`.
- Run `tokenizer.tokenize("Generative AI is transforming e-commerce.")` and print the result — notice which words split into multiple pieces.
- Run the full model on a short prompt and print the generated continuation.

Option B — Ollama (simpler, no Python required)

- Install Ollama and pull a small model, e.g. `ollama pull llama3.2:1b`.
- Run `ollama run llama3.2:1b` and chat with it directly from the terminal.

Optional stretch: visualize attention

Tools like **BertViz** render attention weights as connecting lines between tokens, so you can literally see which words a model is attending to for a given input. Useful for building real intuition beyond the toy bar chart in Section 2.1.

> **Key takeaways**
>
> - Loading a real tokenizer and watching words split into sub-word pieces makes Section 3 concrete.
> - Running a small model end to end (prompt in, tokens out) ties Sections 2 and 4 together.

End of Module 1. Next: Module 2 — Prompt Engineering & LLM APIs.